❸ Returns all the configuration key-value pairs for display purposes

This example added a single JSON configuration provider by calling the `AddJsonFile()` extension method and providing a filename. *It* also set the value of `optional` to `true`, telling the configuration provider to skip files that it can't find at runtime instead of throwing `FileNotFoundException`. When the provider is added, the `ConfigurationManager` requests all the available values from the provider and adds them to the `IConfiguration` implementation.

ConfigurationBuilder vs. ConfigurationManager

Before .NET 6 and the introduction of `ConfigurationManager`, configuration in ASP.NET Core was implemented with `ConfigurationBuilder`. You'd add configuration providers to the builder type the same way you do with `ConfigurationManager`, but the configuration values weren't loaded until you called `Build()`, which created the final `IConfigurationRoot` object.

By contrast, in .NET 6 and .NET 7 `ConfigurationManager` acts as both the builder and the final `IConfigurationRoot`. When you add a new configuration provider, the configuration values are added to the `IConfigurationRoot` immediately, without the need to call `Build()` first.

The `ConfigurationBuilder` approach using the builder pattern is cleaner in some ways, as it has a clearer separation of concerns, but the common use patterns for configuration mean that the new `ConfigurationManager` approach is often easier to use.

If you prefer, you can still use the builder pattern by accessing `WebApplicationBuilder.Host.ConfigureAppConfiguration`. You can read about some of these patterns and the differences between the two approaches on my blog at http://mng.bz/Ke4j.

You can access the `IConfiguration` object directly in Program.cs, as in listing 10.3, but the `ConfigurationManager` is also registered as `IConfiguration` in the dependency injection (DI) container, so you can